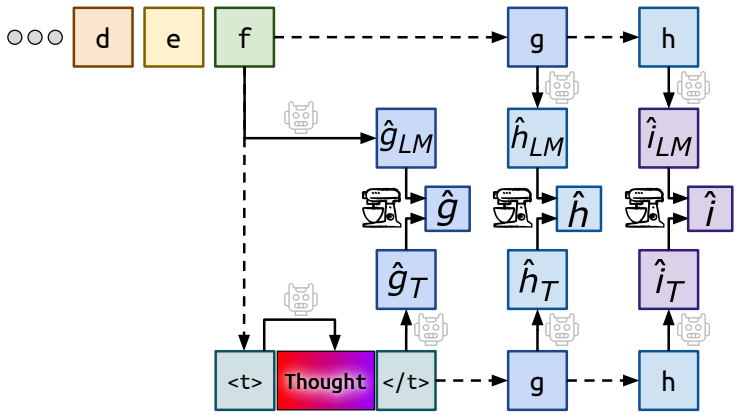

Figure 4: **Forward Pass and Teacher Forcing**. We visualize a single forward pass of our algorithm. Solid lines denote language model computation, while dashed lines indicate tokens are inserted via teacher forcing, and the mixer represents the mixing head. In particular, we visualize predicting three tokens ahead. Thought generation is shown in more detail in Figure 1 and Figure 3.

a given next-token prediction receive a gradient from that prediction—a consequence of our parallel sampling strategy. We could address this by adding loss terms for future tokens by sampling the tokens before. However this would result in much higher entropy for language modeling in general and lower-quality generated text, because it would train the LM to partially disregard its preceding tokens. Instead, we use the parallel attention mask to compute the log probabilities of the true next tokens, applying teacher forcing by assuming the model selected the correct next ground-truth token (as implicit in normal language modeling with transformers). Note that the loss for each future token also depends on a mixing weight computed from the end thought token and the previous observed token. The number of future tokens included in the loss is a hyper-parameter. We apply the same teacher-forcing technique to insert the start and end tokens. We visualize this procedure in Figure 4.

4.4.3 Objective

We use REINFORCE to optimize the likelihoods of the rationales based on their usefullness: the log-likelihood of the n_{true} true next tokens $X_{j+1:j+n_{true}+1}$ under the language model given previous observed tokens and a particular rationale ($p^{\text{talk}}_{j:j+n_{true}}$ as shorthand for the mixed prediction probabilities after thinking, see Algorithm 1). To reduce variance, we generate multiple rationale continuations for each token in the input sequence (loosely inspired by TRICE, Phan et al. (2023)). We thus define the reward r_j for each rationale T_j as the difference between $p^{\text{talk}}_{j:j+n_{true}}$ and the average across rationales for that token ($\overline{p}^{\text{talk}}_{j:j+n_{true}}$):

$$r_j = \log p^{\text{talk}}_{j:j+n_{true}}(X_{j+1:j+n_{true}+1}) - \log \overline{p}^{\text{talk}}_{j:j+n_{true}}(X_{j+1:j+n_{true}+1})$$

We then use this reward in a REINFORCE loss term to update the language model parameters θ to increase the likelihood of rationales that perform better than the average:

$$\nabla_\theta \mathcal{L}^{\text{REINFORCE}}_j = -r_j \cdot \nabla_\theta \log p_\theta(T_j | [X_{:j}; \texttt{<|startofthought|>}])$$

We found it useful to exclude the negative reward from the REINFORCE loss term, as it led to more stable training, though it may introduce some bias.

This loss term encourages the model to generate rationales that improve its predictions of future tokens compared to the average prediction across all generated rationales for that token. The gradients from this loss are used to update both the LM parameters and the start-of-thought and end-of-thought token embeddings, with a (hyperparameter) weight applied to the gradients of the start-of-thought and end-of-thought token embeddings to accelerate their optimization. By iteratively optimizing these parameters, Quiet-STaR trains the model to generate more useful rationales throughout training. Lastly, we also include a log-likelihood loss, $\mathcal{L}^{\text{NLL}}_j$, to ensure that the LM learns to optimize the talking heads and also receives a next-token prediction signal for the base LM head[1].

[1]Due to our linear mixing, equivalent to shifting the mixing weight toward the base prediction.

5 Experiments and Results

Intuitively, not all tokens require equal amounts of thought. For example, consider the sentence "the person is run-": although there is inevitably some probability of the token being something other than "ing"[2], as a standalone sentence without context, additional thinking is unlikely to improve a well-trained model's prediction. Indeed, we conjecture that for most chunks of most online text, additional thought has little to no impact. Indeed, early in our exploration we observed that Quiet-STaR does not benefit all tokens equally. Thus, we design our experiments to investigate whether our approach is useful in predicting tokens that *do* require thought. We evaluate 1) whether Quiet-STaR improves a language model's ability to directly predict answers in datasets that require reasoning; and, 2) the distribution of impacts resulting from thinking tokens. We conduct all of our experiments starting with the base version of Mistral 7B (Jiang et al., 2023).

We perform most of our experiments by training on OpenWebMath (Paster et al., 2023), a crawl that emphasizes more technical webpages. We selected OpenWebMath because we anticipated that it would have a higher density of tokens that benefit from reasoning, which our experiments support. We also evaluate Quiet-STaR on C4 (Raffel et al., 2020), a widely used LM pretraining corpus with more diverse text, and again show significant albeit smaller benefits.

5.1 Downstream Performance

In this subsection, we evaluate the extent to which Quiet-STaR improves the zero-shot reasoning capabilities of the language model on CommonsenseQA (Talmor et al., 2018) and GSM8K (Cobbe et al., 2021). On CommonsenseQA, we find that Quiet-STaR improves performance by 10.9% compared to the base language model. As shown in Figure 2, this improvement consistently increases with the number of tokens used in the model's rationales, indicating that more thorough reasoning through the thought tokens is translating to better direct question-answering performance. Similarly, on GSM8K, Quiet-STaR results in a 5.0% boost over the base model, and once again, performance scales with the length of the rationales generated during Quiet-STaR training. For reference, in Figure 2, we include a baseline corresponding to training the same model on the same dataset without thought tokens. We observe that in multiple curves, performance appears to eventually deteriorate – we anticipate that this is because we are not training on these downstream tasks, so the roles of the thought tokens may change over time. We also find a benefit of our non-myopic objective, which we discuss in Appendix D.

We find that training with Quiet-STaR on C4 (Raffel et al., 2020) also improves performance on GSM8K (5.9% $\rightarrow$ 8.1%) and CommonsenseQA (36.3% $\rightarrow$ 42.6%) but by a smaller margin. Specifically, for our C4 evaluation, we train Mistral 7B with 16 thought tokens and 4 true tokens ahead and otherwise the same setup.

We can compare these improvements to those offered by pause tokens (Goyal et al., 2023), which can be seen as a constrained version of Quiet-STaR where each token is represented by two tokens and the second "pause" token acts as the entirety of the thought. In particular, our setup is most comparable to their pause token fine-tuning, as we also finetune a pretrained model. Their results indicate that pause token fine-tuning also provides minor gains over the base model on CommonsenseQA, they observed an improvement from 26.9% to 28.8%; on GSM8K, Goyal et al. (2023) found that pause token fine-tuning harms performance. Moreover, on both tasks (and the majority of their evaluated tasks), they observed that additional thought tokens harmed performance. Moreover, they discuss the "lukewarm effect of pause-finetuning a standard-pretrained model" (Goyal et al., 2023). This suggests that allowing the model to generate multi-token rationales leads to more effective reasoning compared to the single-token "pauses". Note however, that unlike Goyal et al. (2023), we *do not fine-tune* on the downstream tasks.

Overall, these downstream results validate that training a language model to predict the subtext between the lines of general text data can substantially improve its reasoning

[2]For example, in this very text, the token following "run" is "-"

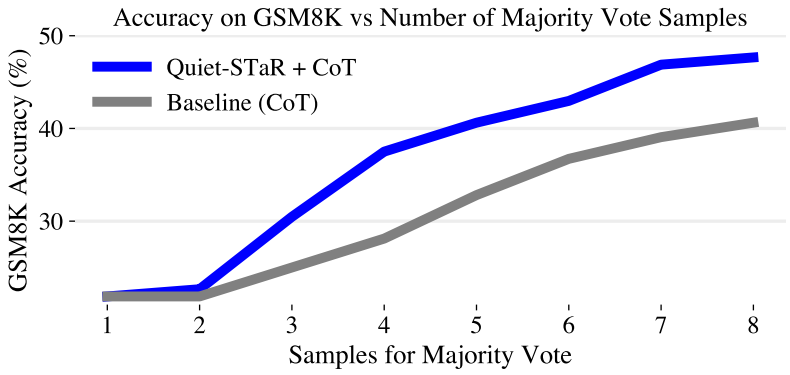

Figure 5: **Zero-shot performance on Quiet-STaR applied to chain-of-thought on GSM8K**. We visualize how using a Quiet-STaR trained Mistral model can improve chain-of-thought performance. We use an 8-thought-token-trained model and use its internal thoughts to improve the tokens in a zero-shot chain-of-thought (Kojima et al., 2022)

capabilities, even on datasets it was not explicitly trained on. The fact that longer rationales consistently lead to better outcomes, and that Quiet-STaR outperforms the constrained pause token approach, supports the notion that Quiet-STaR is successfully teaching the model to leverage its own generated thoughts to reason more thoroughly about the input.

5.2 Improvement Distribution

As visualized in Appendix Figure 7, we find that on average there is little improvement in the LM's ability to predict arbitrary tokens. But, when we visualize the distribution of relative improvements, there is a disproportionate improvement on more difficult tokens. This reflects the idea that some text tokens are substantially harder and benefit more from careful thought.

In Appendix Figure 8, we aim to provide some insight into the kinds of tokens where the improvements occur. Namely, while thinking appears to help for many tokens in the example, inspection suggests it disproportionately help to predict tokens where recalling relevant information is useful, such as the name of an applicable theorem or the start of the next step in a proof. Notably, this would align well with the framing proposed by Prystawski et al. (2024).

5.3 Quiet-STaR and Chain-of-Thought

While there are natural parallels between chain-of-thought prompting and our approach, they are orthogonal and complementary. In zero-shot chain-of-thought, a user actively prompts the model to think 'out loud', otherwise using its ordinary production distribution (Kojima et al., 2022); Quiet-STaR instead allows a model to think quietly at every token, with a distribution trained to be useful. We investigate using silent, Quiet-STaR, rationales while generating explicit CoT reasoning. Because our goal is generalist reasoning that requires no task-specific input at all, we used a zero-shot prompt ("Let's think step by step.") without in-context examples. Our experiments indicate that internal rationales allow the model to generate more structured and coherent chains of thought, shown in Appendix E and visualized in Figure 5. The majority vote accuracy over 8 samples (cot-maj@8) increases from 40.6% to 47.7% with Quiet-STaR, as evaluated on a sample of 128 GSM8K test items. Note that each chain-of-thought solution is sampled with temperature 0.7.

5.4 Examples

While there is no explicit regularization in Quiet-STaR for thoughts to be human-interpretable, they are generated from the same transformer trained to model language, hence likely to be at least partially understandable. We discuss why this design choice benefits the training stability in Appendix I. For reference, we include examples of thoughts generated that were helpful to the model in predicting future tokens in OpenWebMath. First, in one case, recalling that one should start with magnesium to produce magnesium nitride allows it to better predict that the first step of the procedure involves heating magnesium.